

Lecture 5 - Dynamic Analysis (Testing + Fuzzing)

Mục lục

Lecture 5: Dynamic Analysis (Testing, Monitoring, Fuzzing)	4
Vì sao cần Lecture 5 sau khi đã có Lec 3-4?	4
Hai họ kỹ thuật chính của Lecture 5	4
Monitoring	4
Fuzzing	4
Coverage criteria	5
BMC for Test Generation	5
Lộ trình phần	5
Trước khi bắt đầu	5
5.2 Security testing: tại sao khó hơn nhiều?	6
Một bài tập: test đăng nhập	6
Functional vs Security testing	6
Test oracle: vấn đề cốt lõi	6
Khái niệm Test Suite	7
Coverage của Test Suite	7
Các loại Security Testing	7
Vulnerability Scanning	7
Penetration Testing (Pen Test)	7
Security Regression Testing	7
Fuzzing (chi tiết ở bài 5.5-5.7)	8
Property-based Testing	8
Khi nào dùng loại nào?	8
Khó khăn lớn nhất: thinking like attacker	8
Tóm tắt	8
Mini-quiz	8
5.3 Coverage criteria: đo “test đã đủ chưa?”	10
Vấn đề: khi nào test “đủ”?	10
Bốn cấp coverage criteria	10
1. Statement coverage	10
2. Decision coverage (branch coverage)	10
3. Condition coverage	10
4. Modified Condition/Decision Coverage (MC/DC)	11
Bảng so sánh	12
Ví dụ MC/DC chi tiết	12
MC/DC là chuẩn cho avionics	12

Trong thực tế: tool đo coverage	13
Cảnh báo: Coverage không phải mục tiêu	13
Tóm tắt	13
Mini-quiz	13
5.4 Runtime monitoring với LTL và Büchi automata	15
Tại sao cần monitoring khi đã có testing?	15
Linear Temporal Logic (LTL)	15
Ví dụ LTL formula	15
Büchi automaton: machine cho LTL	16
Định nghĩa	16
Ví dụ BA	16
Dùng BA cho runtime monitoring	17
Implementation trên Java với AspectJ	17
Online vs Offline monitoring	18
Soundness của monitoring	18
Generating Büchi automaton từ LTL	18
Tool monitoring trong industry	18
Ứng dụng security	18
Tóm tắt	19
Mini-quiz	19
5.5 Fuzzing basics: ý tưởng đơn giản, sức mạnh không tưởng	20
Một câu chuyện ngắn về Fuzzing	20
Fuzzing pipeline điển hình	20
Random testing thuần (Naive fuzzing)	21
Vì sao fuzzing mạnh đến vậy?	21
Khi nào fuzzing hiệu quả nhất?	21
Tool fuzzing trong thực tế	22
Continuous Fuzzing trong CI	22
Tóm tắt	22
Mini-quiz	22
5.6 Black-box fuzzing: AFL, grammar và mutation	24
Hai trường phái	24
AFL: kẻ đặt nền tảng coverage-guided mutation	24
AFL workflow	24
Ý tưởng 1: Compile-time instrumentation	24
Ý tưởng 2: Forkserver	24
Ý tưởng 3: Energy schedule	26
Ý tưởng 4: Mutation strategy	26
Ưu điểm AFL	26
Nhược điểm AFL	26
AFL++ và biến thể	26
Mutation strategies sâu hơn	26
Bit-level mutation	26
Byte-level mutation	27
Interesting values	27
Dictionary	27

Splice	27
Grammar-based fuzzing	27
Định nghĩa Grammar	27
Tool grammar fuzzer	28
Grammar + mutation hybrid	28
Black-box vs Coverage feedback	28
Khi nào dùng mutation, khi nào grammar?	28
Tóm tắt	28
Mini-quiz	29
5.7 White-box fuzzing: dùng symbolic execution để sinh input	30
Vì sao cần white-box?	30
Dynamic Symbolic Execution (DSE)	30
Ý tưởng	30
Ví dụ chạy DSE	30
DSE vs Symbolic Execution thuần	31
SAGE: white-box fuzzer của Microsoft	31
Architecture	31
Generational search	32
Kết quả thực tế	32
Driller: hybrid AFL + DSE	32
Khó khăn của DSE	32
Path explosion	32
External call	32
Memory model	33
Float	33
DSE tools	33
Tóm tắt	33
Mini-quiz	33
5.8 BMC for test generation: cầu nối ngược về static	35
Vì sao đây là bài cuối Lec 5?	35
Ý tưởng cơ bản	35
Goal 1: reach A ($x < 0$)	35
Goal 2: reach B ($x == 0$)	36
Goal 3: reach C ($x < 100$ nhưng $x \neq 0$ và $x \geq 0$)	36
Goal 4: reach D ($x \geq 100$)	36
Quy trình hình thức	36
Program instrumentation	36
Statement coverage	36
Branch coverage	37
MC/DC	37
So sánh với fuzzing	37
Goal coverage cho complex code	38
Path coverage	38
Ứng dụng thực tế	38
Hạn chế	39
Tóm tắt	39
Mini-quiz	39

Lecture 5: Dynamic Analysis (Testing, Monitoring, Fuzzing)

Tóm tắt một dòng: Khi không chứng minh được “không có bug” bằng formal verification, ta chuyển sang tìm bug bằng cách **chạy chương trình**. Hai họ kỹ thuật chính: **monitoring** theo dõi property tại runtime, và **fuzzing** tự động sinh input để khám phá hành vi. Cuối cùng, hai họ này gặp nhau khi BMC được dùng **ngược** để sinh test case.

Vì sao cần Lecture 5 sau khi đã có Lec 3-4?

Lecture 3 và 4 đã đầu tư rất nhiều vào static analysis: encode chương trình, gọi SMT solver, chứng minh property. Vậy tại sao vẫn cần dynamic?

Vài lý do thực tế:

Thứ nhất, static analysis có giới hạn. Định lý Rice (đã thấy ở [bài 2.1](#)) nói mọi property non-trivial về Turing-complete program đều undecidable. Mọi tool BMC phải trade-off: scope giới hạn, soundness vs completeness, scalability. Có những chương trình mà BMC timeout hoặc không scale tới.

Thứ hai, static analysis phụ thuộc vào model. Tool BMC encode chương trình theo một model nào đó: SC memory model, fixed-size integers, không xét external environment. Bug có thể đến từ chỗ model khác thực tế: một thư viện hệ điều hành cụ thể, một CPU bug, một interrupt timing. Dynamic chạy code thật, bắt được những thứ này.

Thứ ba, dynamic dễ hiểu hơn. Counterexample của BMC là một input trừu tượng, đôi khi khó hiểu. Crash report từ fuzzer kèm core dump, stack trace, có thể debug trực tiếp.

Thứ tư, hai họ bổ sung nhau. Một code base modern dùng BMC verify các module critical (driver, crypto), fuzzer test các thành phần lớn (parser, network protocol), monitor production để bắt bug edge case. Mỗi phần đóng vai trò riêng.

Hai họ kỹ thuật chính của Lecture 5

Monitoring

Monitoring theo dõi chương trình khi nó chạy, check property tại từng bước. Khác fuzzing ở chỗ: monitoring không sinh input, chỉ quan sát chương trình hoạt động (có thể là production traffic hoặc test).

Property cho monitoring thường là **temporal**: “luôn”, “rồi sẽ”, “nếu A thì rồi B”. Diễn đạt bằng **LTL (Linear Temporal Logic)**. Implementation: chuyển LTL formula thành **Büchi automaton**, automaton này chạy song song với chương trình, transition theo event observed.

Tool: AspectJ (Java), DTrace (Solaris/macOS), eBPF (Linux), Splunk monitoring (production).

Fuzzing

Fuzzing sinh input một cách tự động để khám phá hành vi chương trình. Mục tiêu: tìm input gây crash, hang, hoặc vi phạm assertion.

Hai loại lớn:

- **Black-box fuzzing**: không biết code internal, chỉ feed input ngẫu nhiên hoặc theo grammar. Tools: Peach, Sulley.
- **White-box fuzzing**: dùng symbolic execution để sinh input có path coverage cao. Tools: SAGE (Microsoft), KLEE.

Hybrid: **AFL** và **libFuzzer** dùng **coverage-guided mutation**: input nào khám phá branch mới được ưu tiên, mutate tiếp.

Coverage criteria

Để đánh giá fuzzing đã “đủ” hay chưa, dùng coverage criteria: - Statement coverage: mỗi statement chạy ít nhất 1 lần. - Branch coverage: mỗi if/else branch được chọn ít nhất 1 lần. - Condition coverage: mỗi sub-condition được true và false ít nhất 1 lần. - MC/DC (Modified Condition/Decision Coverage): kết hợp condition và decision, tiêu chuẩn cho avionics (DO-178C).

BMC for Test Generation

Cuối cùng, ta dùng BMC **ngược**: thay vì chứng minh property holds, dùng BMC sinh input đạt một goal coverage. Encode goal là một assertion ngược (“không bao giờ đến điểm X”), BMC trả counterexample (input đến X). Counterexample chính là test case.

Ý tưởng này gắn kết Lec 3-4 với Lec 5: cùng một tool BMC, dùng theo hai cách khác nhau cho hai mục tiêu khác nhau.

Lộ trình phần

Bài	Chủ đề	Vì sao cần
5.1	Tổng quan (bài này)	Đặt context
5.2	Security testing	Phân biệt với functional testing
5.3	Coverage criteria	Đo “test đủ chưa”, chuẩn industry
5.4	Monitoring với LTL + Büchi	Theo dõi property tại runtime
5.5	Fuzzing basics	Random testing và giới hạn
5.6	Black-box fuzzing (grammar + mutation, AFL)	Fuzz mà không biết code
5.7	White-box fuzzing (dynamic symbolic execution)	Fuzz với guidance từ code
5.8	BMC for test generation	Cầu nối ngược về static

Trước khi bắt đầu

Phần này giả định bạn đã quen với:

- Khái niệm property, assertion (**bài 2.1**).
- Khái niệm cơ bản BMC, SMT, counterexample (**bài 2.2**).
- (Tuỳ chọn) Quen với LTL từ Logic and Modelling.

Nếu chưa quen LTL, bài 5.4 sẽ giới thiệu lại đủ để hiểu.

Sẵn sàng? Bắt đầu với **bài 5.2: Security testing**.

5.2 Security testing: tại sao khó hơn nhiều?

Tóm tắt một dòng: Functional testing check chương trình **làm đúng cái cần làm** (positive). Security testing check chương trình **không làm cái không nên làm** (negative). Sự khác biệt này, dù tinh tế, dẫn tới những khó khăn fundamental: test oracle không rõ ràng, attack surface rộng, expert kiến thức cần thiết.

Một bài tập: test đăng nhập

Hãy bắt đầu với một ví dụ. Bạn được giao test feature login. Username và password đúng cho user, sai thì reject.

Functional testing tự nhiên: 1. Đăng nhập với credential đúng ☐ kỳ vọng pass, user vào dashboard. 2. Đăng nhập với password sai ☐ kỳ vọng reject, hiện message lỗi. 3. Đăng nhập với username không tồn tại ☐ reject. 4. Đăng nhập với field rỗng ☐ reject.

Đây là 4 test case cover happy path và một số sad path. Trong functional testing, đây có thể đã đủ.

Nhưng từ lăng kính security, còn thiếu rất nhiều:

5. Đăng nhập với username `admin' --` (SQL injection) ☐ kỳ vọng reject, không bypass.
6. Đăng nhập với password có 1 triệu ký tự ☐ kỳ vọng reject hoặc handle gracefully, không crash.
7. Đăng nhập 1000 lần liên tiếp với password sai ☐ kỳ vọng rate limit kick in.
8. Đăng nhập với username có Unicode normalization tricky (`admin` vs `admin` với cyrillic 'a') ☐ kỳ vọng reject hoặc consistent behavior.
9. Đăng nhập rồi check session cookie có `HttpOnly`, `Secure`, `SameSite` không.
10. Đăng nhập trên HTTPS, thử redirect sang HTTP ☐ kỳ vọng không leak session.
11. Đăng nhập trên một browser, dùng cùng session trên browser khác ☐ kỳ vọng có session tracking.
12. ... còn rất nhiều ...

Functional có 4 test, security có 20+. Đây là khoảng cách của hai loại testing.

Functional vs Security testing

Tiêu chí	Functional	Security
Câu hỏi	“Có làm đúng spec không?”	“Có làm sai không (kể cả không trong spec)?”
Test case	Hữu hạn rõ ràng (positive + sad path)	“Vô hạn” (mọi attack vector)
Oracle	“Output đúng spec”	“Không vi phạm security property”
Tester	Developer, QA	Security engineer, pen tester
Tool	Selenium, JUnit	Burp Suite, ZAP, fuzzer

Test oracle: vấn đề cốt lõi

Test oracle là “câu trả lời đúng” mà test so sánh với. Trong functional, oracle thường rõ: spec nói output là gì, test compare.

Trong security, oracle mơ hồ. “Không có bug security” là gì? Không có buffer overflow? Không có SQLi? Không có race? Không có timing attack? Liệt kê hết là bất khả.

Trong thực tế, security testing dùng các oracle gián tiếp: - **Crash**: nếu chương trình crash với input nào, đó là bug (có thể security). - **Sanitizer alert**: ASan, MSan, UBSan, TSan báo memory bug. - **Specific attack succeed**: pen tester thử SQL injection, nếu thành công thì có bug. - **Diff oracle**: chạy 2 version (cũ và mới), nếu output khác có thể regress. - **Differential testing**: chạy 2 implementation tương đương (OpenSSL vs BoringSSL), khác output có thể bug.

Khái niệm Test Suite

Test Suite là tập các test case cùng test cho một mục đích. Trong security:

- **Regression suite**: test các bug đã sửa, đảm bảo không tái phát.
- **Fuzzing corpus**: tập input mẫu dùng làm seed cho fuzzer.
- **Vulnerability scanner check**: rules của tool như Nessus, OpenVAS.
- **Pen test playbook**: các attack chuẩn được tester thử theo thứ tự.

Coverage của Test Suite

Quality của test suite đo bằng coverage. Bài 5.3 đi sâu vào các criterion (statement, branch, MC/DC). Ở đây ta nói tổng quan:

- Coverage cao **không đảm bảo** không có bug, chỉ đảm bảo “code đã được chạm”.
- Coverage thấp **đảm bảo** có chỗ chưa test, có thể có bug.
- Mục tiêu: 80%+ branch coverage là good practice.

Các loại Security Testing

Vulnerability Scanning

Tool tự động scan hệ thống/code tìm các pattern lỗ hổng đã biết. Ví dụ: - **Web scanner**: Nessus, OpenVAS, Burp Suite scanner, OWASP ZAP. Scan URL, gửi payload SQLi/XSS, ghi nhận response. - **Source code scanner (SAST)**: SonarQube, Veracode, Checkmarx. Phân tích source code tìm pattern lỗi. - **Dependency scanner**: Snyk, Dependabot. Check version thư viện có CVE không. - **Container scanner**: Trivy, Clair. Check image Docker có vulnerable package không.

Ưu: nhanh, scale. Nhược: chỉ tìm bug đã biết, miss bug mới.

Penetration Testing (Pen Test)

Manual hoặc semi-automated. Tester đóng vai attacker, thử mọi cách tấn công có thể. Quy trình điển hình:

1. **Reconnaissance**: thu thập thông tin target (subdomain, port mở, công nghệ dùng).
2. **Threat modeling**: liệt kê các attack vector tiềm năng.
3. **Exploitation**: thử exploit, document những gì thành công.
4. **Post-exploitation**: nếu vào được, leo thang quyền, lateral movement.
5. **Reporting**: viết báo cáo cho dev fix.

Pen test đắt nhưng tìm bug sâu hơn scanner.

Security Regression Testing

Khi sửa một bug security, viết test cụ thể tái tạo bug. Bug fix làm test pass. Mọi PR sau chạy test này; nếu fail, regression đã xảy ra.

Best practice modern: mọi CVE filed đều phải có regression test trong codebase.

Fuzzing (chi tiết ở bài 5.5-5.7)

Tự động sinh input, mục tiêu tìm crash. Có thể kết hợp với coverage để hiệu quả hơn.

Property-based Testing

Generalization của fuzzing: định nghĩa property, framework tự sinh input ngẫu nhiên, check property holds. Tool: QuickCheck (Haskell), Hypothesis (Python), proptest (Rust).

Khác fuzzing: PBT có structured input (sinh theo type), check semantic property (không chỉ crash).

Khi nào dùng loại nào?

Không có “best” technique, mọi technique đều có vai trò:

Tình huống	Technique phù hợp
Pre-release scan rộng	Vulnerability scanner
Code mới merge	SAST + dependency scan
Critical release (banking, healthcare)	Pen test
Bug đã sửa	Regression test
Parser, network protocol	Fuzzing
Algorithm correctness	Property-based testing
Production monitoring	Runtime monitor (bài 5.4)

Defense in depth: dùng nhiều technique, mỗi technique bắt một lớp bug.

Khó khăn lớn nhất: thinking like attacker

Cuối cùng, security testing đòi hỏi một **mindset đặc biệt**: nghĩ như attacker. Tester phải tưởng tượng “nếu tôi là kẻ tấn công, tôi sẽ thử gì?”. Đây là kỹ năng cần rèn luyện qua nhiều năm và kiến thức về:

- Các attack pattern phổ biến (OWASP Top 10, CWE Top 25).
- Cách hệ thống thường được bypass.
- Trick về encoding, parsing, race timing.
- Awareness về threat actor (script kiddie vs APT).

Không có tool nào thay thế được mindset này. Đó là lý do pen tester giỏi rất đắt.

Tóm tắt

- Security testing **khó hơn** functional vì test oracle mơ hồ, attack surface rộng.
- **Oracle gián tiếp**: crash, sanitizer alert, specific attack, diff, differential.
- **Test loại**: vulnerability scanner, pen test, security regression, fuzzing, property-based.
- **Defense in depth**: kết hợp nhiều technique.
- **Mindset attacker** không thay thế được bằng tool.

Mini-quiz

Q1. Vì sao test oracle khó định nghĩa cho security?

Functional có spec rõ: “input X $\rightarrow$ output Y”. Oracle là Y, test compare.

Security yêu cầu chương trình “không vi phạm security property”. Property có thể là: - Không có buffer overflow. - Không có SQLi. - Không leak data. - Không bypass auth. - ... vô số ...

Liệt kê hết và check từng cái cho từng input là bất khả. Vì thế dùng oracle gián tiếp: - Crash là dấu hiệu bug (có thể security). - Sanitizer alert là bug được tool detect. - Specific attack succeed là confirmed bug.

Không có oracle “đúng tuyệt đối” cho security testing, chỉ có heuristic bắt được phần lớn bug.

Q2. Phân biệt vulnerability scanning và pen testing.

Vulnerability scanning: - Tự động bằng tool (Nessus, OpenVAS). - Tìm pattern bug đã biết (CVE, CWE Top 25). - Nhanh, scale. - Miss bug mới hoặc bug logic phức tạp.

Penetration testing: - Manual hoặc semi-auto bởi expert. - Tìm bug bằng creative attack, có thể là bug mới. - Đắt (tester \$200-500/giờ, project nhiều ngày). - Tìm bug sâu, đặc biệt là chain attack (bug A leverage bug B).

Best practice: scan trước (cheap, rộng), pen test sau (đắt, sâu) cho critical asset.

Q3. Property-based testing khác fuzzing thế nào?

Fuzzing: input random/mutated, mục tiêu crash. Không có structured property cụ thể. Oracle là “crash hay hang”.

Property-based testing (PBT): định nghĩa property formal (ví dụ “reverse(reverse(list)) == list” hoặc “encrypt then decrypt = identity”). Framework sinh input ngẫu nhiên theo type, check property.

Khác biệt chính: - PBT có structured input (theo type system của ngôn ngữ). - PBT check semantic property (không chỉ crash). - PBT cần programmer viết property (cost effort upfront). - Fuzzing không cần property (chỉ cần input và executable).

Trong thực tế: PBT cho code có spec rõ (algorithm, parser format). Fuzzing cho code thiếu spec hoặc cần test wide.

Modern tool kết hợp cả hai: AFL có thể chạy với assertion (property), Hypothesis (PBT) có coverage-guided mode.

Tiếp theo: [5.3 Coverage criteria](#)

5.3 Coverage criteria: đo “test đã đủ chưa?”

Tóm tắt một dòng: Coverage criteria là tiêu chuẩn đo mức độ “phủ” của test suite trên code. Bốn cấp từ yếu tới mạnh: **Statement** (mỗi dòng), **Decision** (mỗi nhánh if), **Condition** (mỗi sub-condition), **MC/DC** (Modified Condition/Decision). MC/DC là chuẩn bắt buộc trong avionics (DO-178C) vì balance giữa rigor và practicality.

Vấn đề: khi nào test “đủ”?

Một câu hỏi rất thực tế của project manager: “test bao nhiêu là đủ?”. Đáp án naive: “khi nào không còn bug”. Nhưng làm sao biết không còn bug?

Trả lời thực dụng: dùng **coverage criteria**. Coverage đo mức độ “đụng tới” code của test. Coverage cao đảm bảo code đã được chạm, ít bug ẩn. Coverage thấp đảm bảo có chỗ chưa test, có thể có bug ẩn.

Quan trọng: **coverage cao không đảm bảo không bug**. Test có thể chạm code nhưng không trigger bug edge case. Tuy nhiên coverage thấp **đảm bảo** test chưa đủ. Đây là điều kiện cần, không phải đủ.

Bốn cấp coverage criteria

Hãy đi từ yếu tới mạnh.

1. Statement coverage

Định nghĩa: mỗi statement (dòng code thực thi) phải được chạy ít nhất 1 lần bởi test suite.

Ví dụ:

```
void abs(int x) {  
    if (x < 0)  
        x = -x;  
    printf("%d\n", x);  
}
```

Để cover mọi statement: - Statement `if (x < 0)`: bất kỳ test nào cũng chạy. - Statement `x = -x`: cần test với `x < 0`. - Statement `printf`: bất kỳ test nào.

Test suite `{x = -5}` cover hết. Statement coverage = 100%.

Nhưng test này không kiểm tra branch `x >= 0`. Bug có thể ẩn ở đó.

2. Decision coverage (branch coverage)

Định nghĩa: mỗi nhánh của if/else, switch case, while/for condition phải được chọn cả true và false ít nhất 1 lần.

Ví dụ tương tự, để decision cover: - `if (x < 0)` phải có test với `x < 0` (true) và `x >= 0` (false).

Test suite `{x = -5, x = 5}` cover decision. Decision coverage = 100%.

Decision **mạnh hơn** statement: branch false cũng được test.

3. Condition coverage

Định nghĩa: trong compound condition (có `&&`, `||`), mỗi sub-condition phải được true và false ít nhất 1 lần.

Ví dụ:

```
if (a > 0 && b > 0) {
    do_something();
}
```

Để condition cover: - $a > 0$ phải có test true và test false. - $b > 0$ phải có test true và test false.

Test suite: - $\{a = 1, b = 1\}$: $a > 0 = \text{true}$, $b > 0 = \text{true}$. - $\{a = -1, b = -1\}$: $a > 0 = \text{false}$, $b > 0 = \text{false}$.

Cover hết. Nhưng chú ý: $a > 0$ true với $a = 1$, false với $a = -1$. Tương tự b .

Condition coverage **không** đảm bảo decision coverage trong mọi case. Ví dụ trên cover condition nhưng decision chỉ false (cả 2 test đều cho overall expression false, vì $a = 1 \ \&\& \ b = 1$ true thực ra... wait, đó là true). Let me re-check.

$\{a = 1, b = 1\}$: $(a > 0) \ \&\& \ (b > 0) = \text{true} \ \&\& \ \text{true} = \text{true}$. $\{a = -1, b = -1\}$: $\text{false} \ \&\& \ \text{false} = \text{false}$.

Decision: true và false đều có. Decision cover.

Ví dụ tinh tế hơn:

```
if (a > 0 || b > 0) {
    do_something();
}
```

Test suite $\{a = 1, b = -1\}$ ($\text{true} \ || \ \text{false} = \text{true}$) và $\{a = -1, b = 1\}$ ($\text{false} \ || \ \text{true} = \text{true}$).

- $a > 0$: true (test 1), false (test 2). Cover.
- $b > 0$: false (test 1), true (test 2). Cover.

Condition cover. Nhưng decision chỉ true cả 2 lần. Decision không cover false.

Đây là điểm tinh tế: condition coverage **không strictly mạnh hơn** decision coverage.

4. Modified Condition/Decision Coverage (MC/DC)

Định nghĩa: với mỗi sub-condition, phải có cặp test sao cho: - Hai test giống nhau ở mọi sub-condition khác. - Hai test khác nhau ở sub-condition đang xét. - Decision tổng cũng khác nhau.

Nói cách khác, mỗi sub-condition phải **độc lập ảnh hưởng** outcome.

Ví dụ:

```
if (a > 0 && b > 0) {
    do_something();
}
```

Để MC/DC cover: - Sub-condition $a > 0$: - Cặp test ($\{a=1, b=1\}$, $\{a=-1, b=1\}$): a thay đổi, b giữ. Outcome: true $\square$ false. OK. - Sub-condition $b > 0$: - Cặp test ($\{a=1, b=1\}$, $\{a=1, b=-1\}$): b thay đổi, a giữ. Outcome: true $\square$ false. OK.

Cần ít nhất 3 test: $\{a=1, b=1\}$, $\{a=-1, b=1\}$, $\{a=1, b=-1\}$. (Có thể cần 4 cho compound phức tạp.)

Tính chất quan trọng: với n sub-condition, MC/DC cần $n + 1$ test thay vì 2^n (full coverage). Tăng tuyến tính, tractable.

Bảng so sánh

Criterion	Số test (compound 4 sub-conditions)	Rigor	Speed
Statement	~ 1	Yếu	Nhanh
Decision	2	Trung bình	Nhanh
Condition	≤ 8 (mỗi sub $\times 2$)	Trung bình	Nhanh
MC/DC	$\leq 5 (n + 1)$	Mạnh	Trung bình
Multiple Condition (full)	$2^4 = 16$	Rất mạnh	Chậm, không scale

MC/DC là sweet spot: gần “full coverage” về rigor nhưng số test tuyến tính, scale được.

Ví dụ MC/DC chi tiết

Decision: $(A \ \&\& \ B) \ || \ C$.

Mỗi sub-condition (A, B, C) phải độc lập ảnh hưởng outcome. Bảng truth:

Test	A	B	C	(A && B)	(A && B)
T1	T	T	T	T	T
T2	T	T	F	T	T
T3	T	F	T	F	T
T4	T	F	F	F	F
T5	F	T	T	F	T
T6	F	T	F	F	F
T7	F	F	T	F	T
T8	F	F	F	F	F

Để chứng minh A ảnh hưởng outcome: - Cần cặp test có A khác nhau, B và C giữ, outcome khác. - T2 (T,T,F) vs T6 (F,T,F): A khác, B=T, C=F giữ. Outcome T vs F. **OK**.

Để chứng minh B ảnh hưởng outcome: - Cần cặp test có B khác, A và C giữ, outcome khác. - T2 (T,T,F) vs T4 (T,F,F): B khác, A=T, C=F giữ. Outcome T vs F. **OK**.

Để chứng minh C ảnh hưởng outcome: - Cần cặp test có C khác, A và B giữ, outcome khác. - T4 (T,F,F) vs T3 (T,F,T): C khác, A=T, B=F giữ. Outcome F vs T. **OK**.

Cần ít nhất các test: T2, T3, T4, T6. Bốn test cho 3 sub-condition. MC/DC = $n + 1$ trong trường hợp tốt.

MC/DC là chuẩn cho avionics

DO-178C (chuẩn cho phần mềm avionics) yêu cầu MC/DC cho Level A software (catastrophic failure consequences). Các project thực tế:

- Phần mềm điều khiển bay Boeing 787, Airbus A380.
- Hệ thống fly-by-wire F-22.
- ATM, autopilot.

Vì sao MC/DC được chọn? - **Soundness**: nếu MC/DC pass, có rất ít chance miss bug logic. - **Scalability**: tuyến tính theo n , tractable cho code lớn. - **Auditability**: dễ kiểm tra “test này có đạt MC/DC không?”, quan trọng cho certification.

ISO 26262 (automotive) và IEC 61508 (industrial control) cũng tham chiếu MC/DC cho safety-critical level.

Trong thực tế: tool đo coverage

Tool	Ngôn ngữ	Criterion hỗ trợ
gcov	C/C++	Statement, branch, function
lcov	C/C++	Same as gcov, frontend đẹp
JaCoCo	Java	Statement, branch, complexity
coverage.py	Python	Statement, branch
Istanbul/nyc	JS	Statement, branch, function
VectorCAST	C/C++ aviation	MC/DC, certification grade
LDRA	Multi	MC/DC, DO-178C compliant

Hầu hết tool free hỗ trợ statement/branch. MC/DC tool thường commercial (VectorCAST, LDRA), giá hàng chục nghìn USD/seat.

Cảnh báo: Coverage không phải mục tiêu

Có hai sai lầm phổ biến khi đo coverage:

Sai lầm 1: “100% coverage = không bug”. Sai. Coverage chỉ đo code đã được chạm, không đo bug đã được catch. Test “chạm code” nhưng không assert kết quả là vô dụng.

Sai lầm 2: “Coverage thấp □ team kém”. Cũng sai. Coverage thấp có thể vì: - Code có nhiều defensive check (xử lý case hiếm). - Code có integration với external service khó test. - Test viết focus integration thay vì unit.

Coverage là **một** trong nhiều metric, không phải duy nhất. Best practice: theo dõi xu hướng (coverage tăng/giảm theo thời gian), đặt threshold tối thiểu (60-80%), nhưng không obsess.

Tóm tắt

- Coverage criteria đo mức độ phủ test trên code.
- Từ yếu tới mạnh: **Statement, Decision, Condition, MC/DC**.
- **MC/DC** đặc biệt: mỗi sub-condition độc lập ảnh hưởng outcome.
- MC/DC tuyến tính theo n ($n + 1$ test), scale tốt, dùng cho avionics.
- Coverage cao là điều kiện **cần** (không đủ) cho test suite tốt.

Mini-quiz

Q1. Vì sao 100% statement coverage không đảm bảo không có bug?

Statement coverage chỉ check “code đã được chạy”, không check “kết quả đúng”.

Ví dụ:

```
int divide(int a, int b) {
    return a / b;
}
```

Test divide(10, 2) □ 5. Statement cover 100%. Pass.

Nhưng có bug: divide(10, 0) crash (div by zero). Test không cover edge case này.

Cần thêm: - Boundary value test ($b = 0$). - Branch coverage trên các nhánh. - Assertion check result * $b == a$.

Coverage là điều kiện cần (test chạm code), không đủ (test phát hiện bug).

Q2. Vì sao MC/DC tốt hơn full multiple condition cho code có nhiều sub-condition?

Full multiple condition cần test mọi tổ hợp 2^n giá trị. Với $n = 10$, là 1024 test. Với $n = 20$, là 1 triệu test. Không scale.

MC/DC cần $n + 1$ test. Với $n = 10$, là 11 test. Với $n = 20$, là 21 test. Tractable.

MC/DC vẫn đảm bảo mỗi sub-condition **độc lập ảnh hưởng** outcome. Đây là “tinh thần” của full multiple condition (chứng minh logic của mỗi sub không thừa), đạt được với chi phí tuyến tính thay vì mũ.

Đây là lý do MC/DC được DO-178C chọn: rigor cao + scale.

Q3. Tính MC/DC test cho decision $(A \parallel B) \&\& C$.

Truth table:

A	B	C	$(A \parallel B)$	$(A \parallel B) \&\& C$
T	T	T	T	T
T	T	F	T	F
T	F	T	T	T
T	F	F	T	F
F	T	T	T	T
F	T	F	T	F
F	F	T	F	F
F	F	F	F	F

Để A độc lập ảnh hưởng: - B và C giữ, A thay đổi, outcome khác. - Tìm cặp: B=F, C=T. A=T: outcome T. A=F: outcome F. Cặp T3 vs T7. **OK**.

Để B độc lập ảnh hưởng: - A và C giữ, B thay đổi. - A=F, C=T. B=T: outcome T. B=F: outcome F. Cặp T5 vs T7. **OK**.

Để C độc lập ảnh hưởng: - A và B giữ, C thay đổi. - Bất kỳ A, B làm $(A \parallel B) = T$. Lấy A=T, B=F. C=T: T. C=F: F. Cặp T3 vs T4. **OK**.

Test set: T3, T4, T5, T7. **4 test** cho 3 sub-condition. Đúng $n + 1$.

DS perspective

Coverage criteria trong testing trả lời “tôi đã test bao nhiêu phần code?” giống **k-fold cross-validation** trả lời “tôi đã đánh giá trên bao nhiêu phần data?”. MC/DC đặc biệt giống **factorial design** trong DOE / ANOVA: với n binary feature, cần $\geq n + 1$ test để cover mọi pair interaction. **Branch coverage** $\approx$ **stratified sampling** (mỗi class được cover). **Uncovered branch** $\approx$ **underrepresented subgroup** trong training set - cùng là red flag về data/test quality. Nếu bạn từng tune model với stratified k-fold, bạn đã có instinct về MC/DC.

Tiếp theo: [5.4 Monitoring với LTL và Büchi automata](#)

5.4 Runtime monitoring với LTL và Büchi automata

Tóm tắt một dòng: Runtime monitoring là kỹ thuật theo dõi chương trình **khi nó chạy**, check property tại từng event. Property thường là temporal (liên quan thứ tự thời gian), diễn đạt bằng **LTL**. Implementation: chuyển LTL sang **Büchi automaton**, chạy automaton song song với chương trình, alert nếu vào state “vi phạm”.

Tại sao cần monitoring khi đã có testing?

Testing chạy chương trình với input cụ thể trong môi trường test. Monitoring chạy chương trình trong **môi trường thực** (production, staging), với input thực, observed.

Khác biệt:

Tiêu chí	Testing	Monitoring
Môi trường	Test, isolated	Production, real
Input	Cụ thể, chuẩn bị trước	Real-time, không control
Mục tiêu	Tìm bug trước khi release	Phát hiện sự cố trong production
Action khi fail	Block release	Alert ops, có thể self-heal

Trong DevOps modern, monitoring là **lớp cuối** của defense in depth. Sau khi test pass, monitor production để bắt bug edge case mà test miss.

Property monitoring trong code thường có dạng temporal:

- “Mọi request đều có response trong 5 giây.”
- “Sau khi user login, mọi action đều có authentication.”
- “Khi malloc fail, không có dereference của pointer đó.”
- “Mọi connection cuối cùng đều được close.”

Diễn đạt các property này cần một **ngôn ngữ thời gian**, đó là LTL.

Linear Temporal Logic (LTL)

LTL là logic với các **temporal operator** ngoài boolean thông thường:

Operator	Đọc	Nghĩa
$G\phi$	Globally ϕ	ϕ đúng tại mọi thời điểm
$F\phi$	Finally ϕ	ϕ đúng tại một thời điểm trong tương lai
$X\phi$	Next ϕ	ϕ đúng tại thời điểm kế tiếp
$\phi U \psi$	ϕ Until ψ	ϕ đúng cho tới khi ψ đúng

Kết hợp boolean ($\wedge$, $\vee$, $\neg$) và temporal, ta diễn đạt được hầu hết property thực tế.

Ví dụ LTL formula

“Safety: không bao giờ chia cho 0”:

$$G\neg(\text{div_op} \wedge \text{denominator} = 0)$$

“Liveness: mọi request rồi sẽ có response”:

$$G(\text{request} \implies F\text{response})$$

“Sau lock phải có unlock trước khi lock lại”:

$$G(\text{lock} \implies X(\neg\text{lock}U\text{unlock}))$$

“Sau khi malloc, không free trước khi dùng” (xấp xỉ):

$$G(\text{malloc}(p) \implies (\text{not free}(p))U\text{use}(p))$$

LTL là powerful: diễn đạt được hầu hết safety và liveness property mà ta gặp trong practice.

Büchi automaton: machine cho LTL

Để implement runtime monitor, ta cần “biến” LTL formula thành thứ có thể chạy. Cách phổ biến: dịch sang **Büchi automaton (BA)**.

Định nghĩa

Büchi automaton là finite-state automaton trên infinite word. Cấu trúc:

- Q : tập state.
- Σ : alphabet (tập event).
- $\delta : Q \times \Sigma \rightarrow 2^Q$: transition.
- $q_0 \in Q$: initial state.
- $F \subseteq Q$: accepting state.

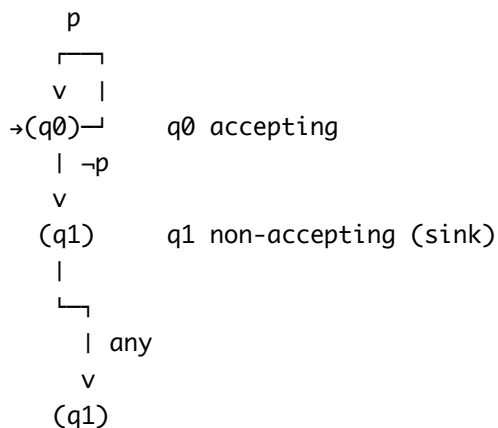
Khác automaton thông thường: BA accept **infinite word** $w = a_1a_2a_3 \dots$ nếu có run vô hạn đi qua một state trong F **vô hạn lần**.

Định lý: với mỗi LTL formula ϕ , tồn tại Büchi automaton $\mathcal{A}_\phi$ accept đúng các infinite word thoả ϕ .

Ví dụ BA

LTL: Gp (“ p luôn đúng”).

BA tương ứng có 1 state, là accepting, transition tự loop với event p . Nếu gặp $\neg p$, transit sang state non-accepting (sink). Word vô hạn chỉ accept nếu mọi event là p .



LTL: Fp ("rồi p sẽ đúng").

BA có 2 state: q_0 (chưa gặp p), q_1 (đã gặp p). q_1 accepting. Transition: q_0 với $\neg p \rightarrow q_0$; q_0 với $p \rightarrow q_1$; $q_1 \rightarrow q_1$.

Word infinite cần accept khi cuối cùng gặp p rồi stay tại q_1 mãi (đi qua q_1 vô hạn lần).

Dùng BA cho runtime monitoring

Algorithm:

```
def monitor(events_stream, ba):
    state = ba.initial_state
    for event in events_stream:
        state = ba.transition(state, event)
        if state == sink_state:
            alert("Property violated")
            break
```

Mỗi event từ chương trình (function call, variable update, network message), monitor transit BA. Nếu vào sink state (bad), alert.

Đây là **online monitoring**: process event real-time, không cần lưu toàn bộ trace.

Implementation trên Java với AspectJ

```
@Aspect
public class MutexMonitor {
    private State currentState = State.UNLOCKED;

    @Before("execution(* lock(..))")
    public void onLock() {
        if (currentState == State.LOCKED) {
            // Vi phạm: đã lock, lại lock nữa (double lock)
            throw new RuntimeException("Property violation: double lock");
        }
        currentState = State.LOCKED;
    }

    @Before("execution(* unlock(..))")
    public void onUnlock() {
        if (currentState == State.UNLOCKED) {
            throw new RuntimeException("Property violation: unlock without lock");
        }
        currentState = State.UNLOCKED;
    }
}
```

Đây là BA đơn giản với 2 state (LOCKED, UNLOCKED), implement bằng aspect. Tự động chạy trước mỗi `lock()` và `unlock()` trong codebase.

Online vs Offline monitoring

Online monitor: chạy đồng thời với chương trình. Alert real-time. Phù hợp production. Overhead vài %.

Offline monitor: log mọi event ra file, chạy monitor trên file sau khi chương trình kết thúc. Phù hợp post-mortem analysis. Không có overhead runtime nhưng cần I/O lớn.

Một số tool support cả hai: log everything in production, replay với BA monitor khi cần.

Soundness của monitoring

Vấn đề: BA có chỉ chứng minh được safety property (counterexample hữu hạn). Liveness property cần infinite trace, monitor không thể “đợi mãi”.

Giải pháp: - **Bounded liveness:** thay $F\phi$ bằng “ ϕ xảy ra trong k event tới”. Hữu hạn, monitor được. -

Robust liveness: định nghĩa “vi phạm liveness” là “đã k event mà ϕ chưa xảy ra”.

Hầu hết production monitor dùng bounded liveness (k là timeout).

Generating Büchi automaton từ LTL

Manual: cho LTL nhỏ, vẽ BA tay. Đã thấy ví dụ Gp , Fp .

Automated: tool **LTL2BA**, **Spot**. Input là LTL formula, output là Büchi automaton.

Algorithm gốc: **Vardi-Wolper construction**. Build BA exponential trong size LTL formula. Modern optimization (alternating automata, generalized BA) giảm size đáng kể.

Trong thực tế, code monitoring thường có LTL formula nhỏ (1-2 operator), BA chỉ vài state, dễ implement tay.

Tool monitoring trong industry

AspectJ (Java): aspect-oriented programming, dễ inject monitor.

DTrace (Solaris, macOS, FreeBSD): probe-based, kernel + userland.

eBPF (Linux modern): runtime kernel program, monitor mọi syscall, network event. Cực kỳ powerful.

Splunk: enterprise log monitoring, có thể implement BA qua query.

Prometheus + Grafana: metric-based monitor, alert qua threshold.

Datadog APM: trace-based, application performance monitoring với security extension.

Ứng dụng security

Runtime monitor cho security:

Anomaly detection: monitor mọi syscall, alert nếu có syscall lạ (process tạo network connection bất thường).

File integrity monitoring (FIM): monitor `/etc/passwd`, `/etc/shadow`. Alert nếu sửa.

Authentication monitoring: count fail login, alert nếu vượt threshold (brute force).

Privilege escalation detection: monitor `execve` của `setuid` binary, alert nếu user gọi không đúng context.

Network anomaly: monitor traffic pattern, alert outbound to suspicious IP.

Combine với SIEM (Security Information and Event Management) như Splunk, ELK, để có visibility toàn bộ infrastructure.

Tóm tắt

- **Monitoring** check property tại runtime, bổ sung testing/verification.
- **LTL** diễn đạt temporal property (safety và liveness).
- **Büchi automaton** là machine để implement LTL monitor, chạy song song với chương trình.
- **Online** monitor real-time, **offline** monitor log replay.
- Tools: AspectJ, DTrace, eBPF, Splunk, Prometheus.
- Ứng dụng security: anomaly detection, FIM, authentication monitoring.

Mini-quiz

Q1. Phân biệt safety và liveness trong context monitoring.

Safety (“điều xấu không xảy ra”): counterexample hữu hạn. Khi vi phạm xảy ra, monitor phát hiện ngay tại event đó. Ví dụ: “không double lock” - khi gặp lock lần 2 mà đã lock, alert ngay.

Liveness (“điều tốt sẽ xảy ra”): counterexample vô hạn. Monitor không thể “đợi mãi” để chứng minh liveness violation. Cần **bounded liveness**: timeout sau k event.

Ví dụ: “request rồi sẽ có response” - không thể đợi vô hạn. Bounded: “request có response trong 5 giây”. Alert nếu sau 5 giây không response.

Hầu hết production monitor dùng bounded liveness vì soundly chạy được.

Q2. Tại sao Büchi automaton accept infinite word thay vì finite word?

LTL property thường liên quan execution **vô hạn** (chương trình chạy forever, như daemon, server). Property Gp (“luôn p ”) cần xem toàn bộ infinite trace.

Finite-word automaton (DFA, NFA) không đủ: chỉ accept/reject finite word.

Büchi giải quyết: accept infinite word nếu có run đi qua accepting state **vô hạn lần**. Vì state hữu hạn nhưng word vô hạn, run phải lặp lại state. Lặp trên accepting $\square$ accept.

Trong monitoring, ta chỉ process prefix hữu hạn của infinite trace (chương trình chưa kết thúc). BA phát hiện vi phạm khi vào sink state (bad state không escape được), không cần đợi infinity.

Q3. eBPF có gì đặc biệt cho monitoring so với DTrace?

DTrace (1990s, Solaris): probe-based, có ngôn ngữ D để write monitor script. Portable cho macOS, FreeBSD. Linux có port nhưng chưa được adopt rộng.

eBPF (2014, Linux): tương tự về concept (probe + script), nhưng: - **Native trên Linux**: không cần kernel patch, ship sẵn. - **JIT compile**: script eBPF được compile thành native code, chạy in kernel space, rất nhanh. - **Safe**: verifier check script không crash kernel trước khi load. - **Versatile**: probe syscall, network, file system, even userland (uprobes).

Modern security tool dùng eBPF: Falco (Sysdig), Cilium (CNCF), Tetragon. Hầu hết observability stack hiện đại trên Linux đều base on eBPF.

Trade-off: eBPF require Linux 4.x+, không portable. DTrace portable hơn nhưng chậm hơn.

Tiếp theo: [5.5 Fuzzing basics](#)

5.5 Fuzzing basics: ý tưởng đơn giản, sức mạnh không tưởng

Tóm tắt một dòng: Fuzzing là kỹ thuật tự động sinh input ngẫu nhiên (hoặc semi-random) để đẩy cho chương trình, mục tiêu tìm crash, hang hay vi phạm assertion. Đơn giản đến nỗi sinh viên đại học cũng implement được, nhưng đã tìm hàng nghìn CVE trong các phần mềm quan trọng nhất thế giới.

Một câu chuyện ngắn về Fuzzing

Năm 1989, giáo sư Barton Miller tại University of Wisconsin nhận xét: trong một cơn bão, terminal của ông bị nhiễu, gửi ký tự ngẫu nhiên vào Unix shell. Shell crash. Ông nghĩ: “nếu nhiễu ngẫu nhiên crash shell, vậy mọi Unix utility có chịu được random input không?”

Ông tạo một tool đơn giản: sinh chuỗi byte ngẫu nhiên, feed cho mọi command line Unix utility (ls, grep, awk, ...). Kết quả gây sốc: **25-33% utility crash** trong 5 phút testing. Bug từ năm 1980s vẫn chưa được phát hiện.

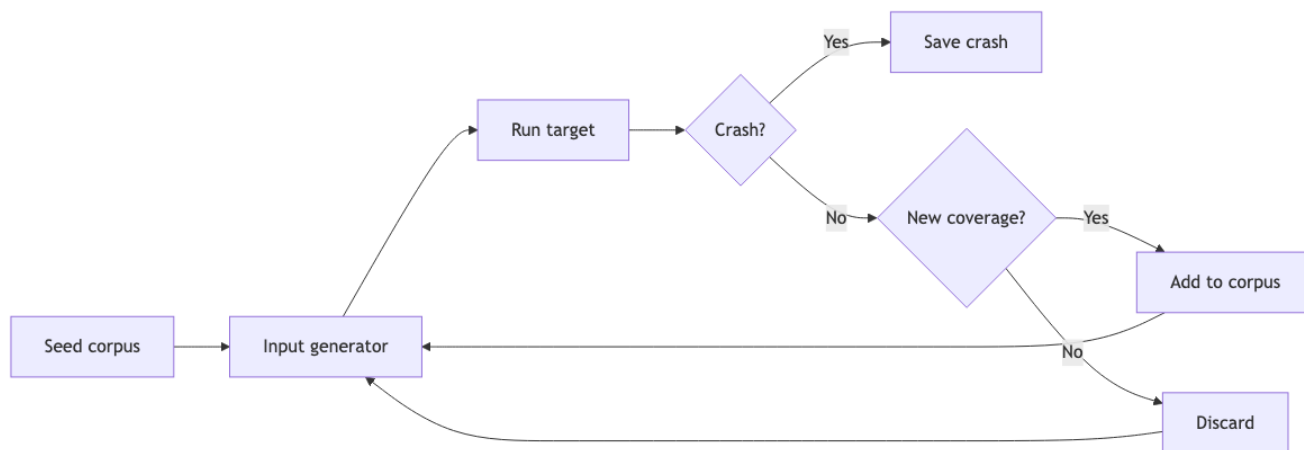
Đây là khởi nguồn của **fuzzing**. Từ “fuzz” lấy từ trên đường (radio fuzz = nhiễu).

35 năm sau, fuzzing đã phát triển thành một ngành lớn. Google **OSS-Fuzz** chạy fuzzer 24/7 trên 500+ open-source project, tìm 60,000+ bug từ 2016. Microsoft **OneFuzz** tương tự cho code Microsoft. Hầu hết bug Heartbleed-tâm-cổ trong thập kỷ qua đều được phát hiện bởi fuzzer chứ không phải pen tester hay code review.

Tại sao đơn giản nhưng hiệu quả? Vì code thực tế **đầy edge case** mà developer không nghĩ tới khi viết. Random input chạm vào edge case nhanh hơn dù không có pattern.

Fuzzing pipeline điển hình

Pipeline fuzzing 4 bước:



Hình 1: Sơ đồ 8

Seed corpus: tập input ban đầu, “good examples” mà developer cung cấp. Ví dụ với PDF fuzzer, seed là vài PDF hợp lệ.

Input generator: từ corpus hiện tại, sinh input mới. Hai chiến lược: - **Generation-based:** sinh input từ scratch theo grammar/template. - **Mutation-based:** lấy input từ corpus, “mutate” (flip bit, insert bytes, delete) tạo input mới.

Run target: chạy chương trình với input. Có thể là binary, library, network service. Phải instrument để detect crash.

Crash detection: native crash (segfault, abort), assertion failure, sanitizer alert (ASan, MSan).

Coverage feedback: nếu input mới khám phá branch/path chưa từng thấy, add vào corpus. Loop tiếp.

Random testing thuần (Naive fuzzing)

Cách đơn giản nhất: sinh input hoàn toàn ngẫu nhiên, không nhìn lại code.

```
import random
import subprocess

while True:
    n = random.randint(1, 10000)
    data = bytes(random.randint(0, 255) for _ in range(n))
    result = subprocess.run(['./target'], input=data, timeout=5)
    if result.returncode != 0:
        save_crash(data)
```

Đơn giản 10 dòng code. Đây là cách Miller thí nghiệm 1989.

Ưu: cực kỳ dễ implement. Không cần biết internals.

Nhược: rất chậm tìm bug “deep” cần input có cấu trúc. Ví dụ JSON parser, random byte hầu như chắc chắn fail ở parse level đầu tiên. Bug ở deep logic (sau khi parse xong) không bao giờ được test.

Random testing có giá trị làm baseline, nhưng modern fuzzing dùng kỹ thuật smarter.

Vì sao fuzzing mạnh đến vậy?

Có nhiều giả thuyết về sự thành công của fuzzing:

Giả thuyết 1: Code có nhiều edge case “không suy nghĩ”. Developer test happy path và một vài sad path. Code phải handle hàng triệu edge case (empty input, huge input, malformed byte at offset X, unicode normalization, integer overflow trong size calculation, ...). Random input chạm các edge case đó.

Giả thuyết 2: Memory bug rất phổ biến trong C/C++. Buffer overflow, use after free, double free, ... 70% CVE trong Chromium, Linux kernel là memory bug. Fuzzer + sanitizer phát hiện 99% memory bug trong phần code đã chạm.

Giả thuyết 3: Coverage tự động cập nhật. Coverage-guided fuzzing (AFL) tự learn từ chương trình mà không cần human guidance. Càng chạy lâu, càng tốt.

Giả thuyết 4: Parallel scaling. Fuzzer có thể chạy 1000 instance song song trên cloud. Cost vài USD/giờ cho 1000 core, tìm được bug mà 1 pen tester tìm cả tháng không ra.

Khi nào fuzzing hiệu quả nhất?

Fuzzing hoạt động tốt cho:

Parsers: file format (PDF, image, video), network protocol (HTTP, DNS), serialization (JSON, XML). Mọi parser từng được fuzz đều tìm thấy bug.

Stateless code: function thuần túy (hash, encrypt, compress). Easy to fuzz, no setup needed.

Crypto primitive: implementation detail của AES, RSA, hash. Fuzzer tìm side-channel, edge case về key derivation.

Sandbox boundary: JIT compiler, eBPF verifier, Wasm runtime. Bug ở đây có security impact lớn.

Kém hiệu quả cho:

Stateful service với deep state machine: cần input theo chuỗi để vào state interesting. Random byte không đủ.

GUI / event-driven: cần simulate user action, không phải input file.

Concurrent code: thread interleaving không tự nhiên trong fuzzing. Cần concurrency-aware fuzzer.

Tool fuzzing trong thực tế

AFL (American Fuzzy Lop, 2013): coverage-guided mutation-based. Có lẽ là fuzzer ảnh hưởng nhất. Tìm hàng nghìn bug trong OpenSSL, sqlite, libpng. Chi tiết ở bài 5.6.

AFL++: fork của AFL với nhiều cải tiến. Active maintain.

libFuzzer: in-process fuzzer của LLVM/Clang. Faster than AFL vì không fork process. Phù hợp cho library code.

Honggfuzz: Google's fuzzer, hỗ trợ AFL-style + perf counter feedback.

syzkaller: Google's kernel fuzzer. Tìm hàng nghìn bug trong Linux kernel.

OSS-Fuzz: platform của Google chạy fuzzer trên open-source project. Free tier cho project bất kỳ.

peach: commercial generation-based fuzzer cho network protocol.

Continuous Fuzzing trong CI

Modern practice: fuzz trong CI pipeline, không phải one-off.

ClusterFuzz (Google): chạy fuzzer 24/7 trên cluster. Khi tìm bug, auto file ticket, assign developer.

OSS-Fuzz integration: project có fuzz target được Google fuzz miễn phí trên cluster của họ. Bug fixing dưới 30 ngày.

Coverage growth tracking: theo dõi coverage qua thời gian. Coverage stagnate = fuzzer stuck, cần thêm seed corpus hoặc cải tiến grammar.

Tóm tắt

- **Fuzzing** sinh input tự động để tìm crash/hang.
- Lịch sử: Miller (1989) random byte □ modern AFL (2013) coverage-guided.
- Đơn giản nhưng hiệu quả nhờ code có nhiều edge case không suy nghĩ.
- Hiệu quả nhất cho parser, stateless code, crypto primitive.
- Tool chính: AFL, libFuzzer, syzkaller, honggfuzz.
- Modern: continuous fuzzing trong CI với ClusterFuzz, OSS-Fuzz.

Mini-quiz

Q1. Random testing thuần và coverage-guided fuzzing khác nhau thế nào?

Random testing thuần: sinh input hoàn toàn ngẫu nhiên, không feedback từ chương trình. Mỗi iteration độc lập.

Coverage-guided fuzzing: lấy seed từ corpus, mutate. Sau khi chạy, check coverage. Nếu input mới khám phá branch chưa thấy, **giữ lại trong corpus**. Lần mutate sau lấy seed này (đã thấy branch mới), có cơ hội đi sâu hơn.

Ví dụ: parser cần password đầu file để vào critical code. Random testing chắc gần như không bao giờ sinh đúng password. Coverage-guided: một input ngẫu nhiên may mắn pass password check → vào critical code → coverage tăng → giữ input. Mutate input đó cho lần sau, “memorize” password rồi từ từ explore critical code.

Coverage-guided thắng random testing ở các bug “deep” (cần input cấu trúc).

Q2. Vì sao Miller 1989 phát hiện ra 25-33% Unix utility crash với random input?

Có nhiều lý do:

Lý do 1: Unix utility được viết trong 1970s-1980s khi chưa có culture defensive coding. Code giả định input “reasonable” (có newline ở cuối, không quá dài, ...).

Lý do 2: C language không có bounds check. Buffer overflow, integer overflow rất dễ xảy ra với input lạ.

Lý do 3: Hệ thống file/process model phức tạp. Argv quá dài, stdin có byte NUL ở giữa, file descriptor close giữa chừng - đủ thứ tình huống developer không test.

Lý do 4: Không có tool fuzzing trước Miller. Bug từ thập kỷ 80 nằm trong code không ai thử.

Nhưng quan trọng nhất: random byte chạm các edge case mà developer không nghĩ tới. Đây là insight gốc của fuzzing, vẫn đúng ngày nay.

Q3. Tại sao fuzzing không hiệu quả cho GUI testing?

GUI có hai thách thức:

Input phức tạp hơn byte: input GUI là sequence event (click, key, drag) với timing. Random click vô nghĩa không trigger flow logic.

State machine sâu: để test feature ở 10 click deep, fuzzer cần sinh 10 click đúng theo flow. Random click hầu như chắc chắn đi lạc.

Cần **GUI-aware fuzzer**: hiểu DOM/widget tree, biết click vào button nào dẫn tới dialog nào. Tool như **Sapienz** (Facebook) dùng evolutionary algorithm + GUI awareness để fuzz mobile app.

Trong web context: **Burp Suite** + custom extension có thể “fuzz form” theo flow login → action → submit. Tốt hơn random nhưng vẫn không gần độ hiệu quả của file format fuzzing.

DS perspective

Fuzzing **chính là** adversarial example search nhưng cho bug thay vì cho misclassification.

Seed corpus ≈ training data, **mutation strategy** ≈ data augmentation (flip, rotate, noise).

Coverage-guided fuzzing (AFL) ≈ **active learning** - chọn input most informative (cover code mới). **AFL energy schedule** ≈ **curriculum learning**. **Crash** ≈ misclassification, **counterexample** ≈ adversarial example. Nếu bạn từng dùng cleverhans hay foolbox cho adversarial ML, fuzzing intuitive ngay - cùng là “search trong input space để tìm fail mode”.

Tiếp theo: [5.6 Black-box fuzzing \(AFL, grammar, mutation\)](#)

5.6 Black-box fuzzing: AFL, grammar và mutation

Tóm tắt một dòng: Black-box fuzzing sinh input **mà không phân tích source code** (chỉ binary và optionally coverage instrument). Hai chiến lược chính: **mutation-based** (AFL) lấy seed corpus mutate ngẫu nhiên với feedback coverage; **grammar-based** sinh input theo grammar/template formal. AFL là king cho code C/C++ không cấu trúc cao, grammar là king cho input structured (network protocol, JSON, SMT).

Hai trường phái

Black-box fuzzing có hai cách “sinh input”:

Mutation-based: bắt đầu từ seed corpus (input mẫu hợp lệ), apply mutation (flip bit, insert bytes, delete byte) để tạo input mới. Mutation random hoặc có heuristic.

Generation-based (a.k.a. grammar-based): không cần seed, sinh input từ scratch theo grammar formal mô tả format.

Hai trường phái không loại trừ nhau. Modern fuzzer thường combine: grammar để sinh seed initial, mutation để explore variant.

AFL: kẻ đặt nền tảng coverage-guided mutation

AFL (American Fuzzy Lop), 2013 bởi Michał Zalewski, là coverage-guided mutation-based fuzzer. AFL đã trở thành “gold standard” mà mọi fuzzer modern so sánh với.

AFL workflow

Bốn ý tưởng cốt lõi:

Ý tưởng 1: Compile-time instrumentation

AFL không observe coverage qua debugger (chậm). Thay vào đó, compile target với AFL compiler (afl-gcc, afl-clang). Compiler chèn instrumentation code vào mọi basic block:

```
// Mỗi basic block sau instrumentation:
void original_basic_block() {
    afl_track_branch(BLOCK_ID); // <-- added by AFL compiler
    // ... original code ...
}
```

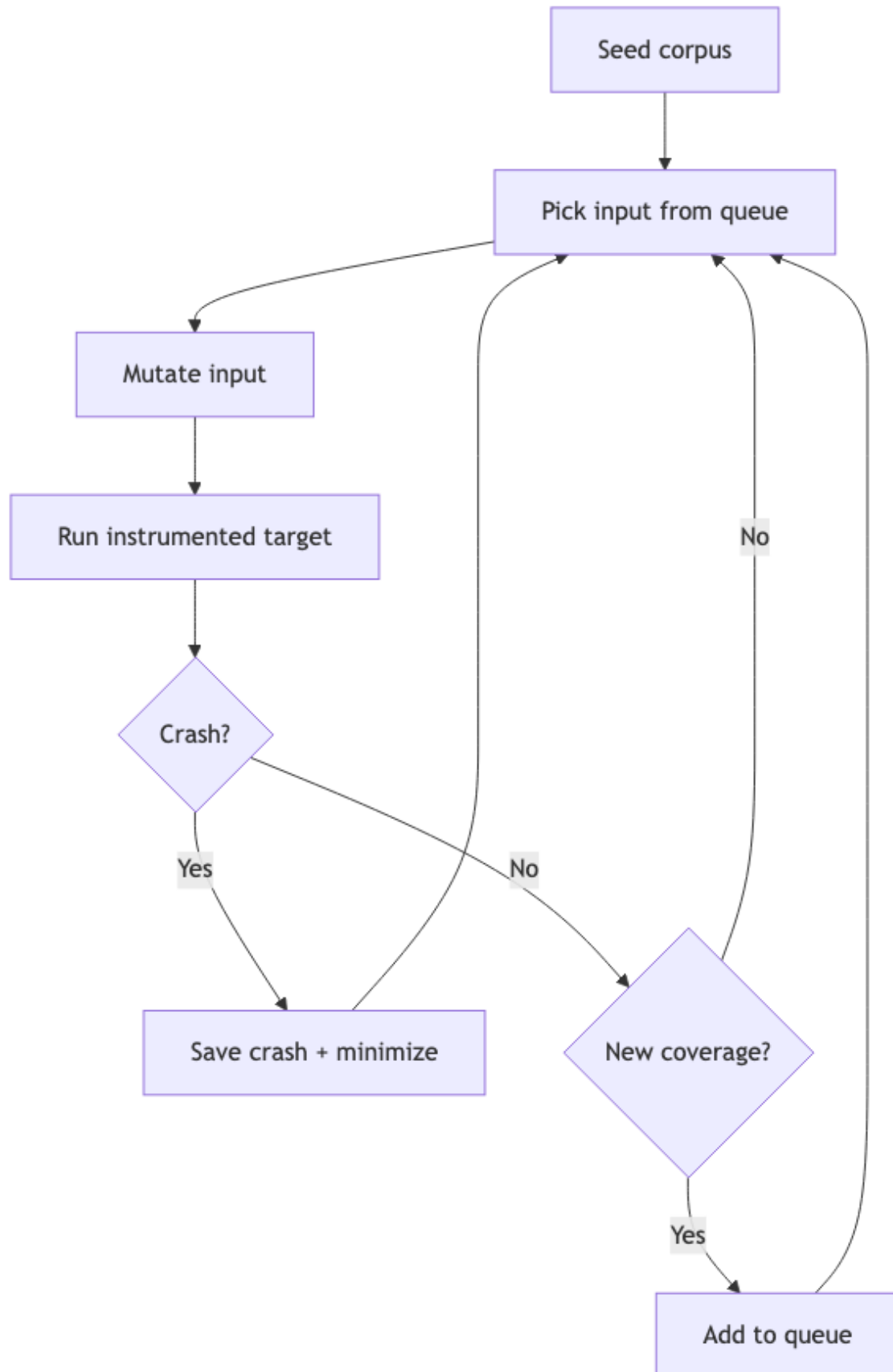
afl_track_branch ghi vào shared memory map: “đã đi qua từ block A sang block B”. Map này gọi là **AFL bitmap**, kích thước 64KB.

Sau khi target chạy, AFL đọc bitmap, so sánh với bitmap “đã thấy”. Nếu có entry mới (transition mới), input này có coverage mới.

Ý tưởng 2: Forkserver

AFL không exec target mỗi lần. Thay vào đó dùng **forkserver**: target được start một lần, mỗi test case fork một process con, exec input, exit. Tiết kiệm cost loader, dynamic linker.

Trên Linux modern, AFL có thể chạy 1000-10000 test/giây trên 1 core.



Ý tưởng 3: Energy schedule

AFL không treat mọi seed bằng nhau. Mỗi seed có “energy” = số mutation áp dụng. Energy cao cho seed:
- Mới (chưa được mutate nhiều). - Đến từ chỗ “rare” của coverage map. - Nhỏ và nhanh.

Algorithm: weighted random pick từ queue.

Ý tưởng 4: Mutation strategy

AFL không mutate ngẫu nhiên thuần. Có chiến lược layered:

1. **Bit flip**: lật 1 bit, 2 bit, 4 bit liên kề.
2. **Byte flip**: lật 1 byte.
3. **Arithmetic**: cộng/trừ giá trị nhỏ vào byte/word/dword.
4. **Interesting values**: thay byte bằng giá trị “interesting” (0, 1, -1, INT_MAX, ...).
5. **Dictionary**: insert keyword từ dictionary (token, magic number).
6. **Havoc**: random combination của trên.
7. **Splice**: ghép 2 seed thành 1.

Mỗi strategy thử cho mỗi seed, ưu tiên strategy chưa thử.

Ưu điểm AFL

- **Bug detection**: tìm hàng nghìn CVE trong OpenSSL, libpng, sqlite, libxml2, ...
- **Generic**: chạy được cho mọi target có instrument được.
- **Self-tuning**: energy schedule + coverage feedback tự động improve.
- **Open source**, miễn phí, mature.

Nhược điểm AFL

- **Chậm cho deep coverage**: cần thời gian dài (giờ, ngày) để khám phá branch sâu.
- **Khó cho input structured**: AFL random mutation hỏng input format ngay (JSON parser, AFL mutation tạo invalid JSON).
- **Phụ thuộc seed corpus**: corpus thiếu diversity $\square$ coverage thấp.

AFL++ và biến thể

AFL++ (2019): fork với nhiều cải tiến: better mutator (CmpLog), better energy schedule, parallel modes. Tốc độ tìm bug 2-5x AFL gốc.

libFuzzer (LLVM): in-process variant. Faster cho library code (không fork overhead).

Honggfuzz: tương tự AFL, hỗ trợ Linux perf counter (CPU branch counter).

Mutation strategies sâu hơn

Bit-level mutation

Thay đổi bit pattern. Ví dụ input 0x41 (ASCII ‘A’): - Flip bit 0: 0x40 (ASCII ‘@’). - Flip bit 7: 0xC1 (high bit set).

Hữu ích cho bug ở mức bit (bit field, packed struct).

Byte-level mutation

Random byte. Insert byte ở vị trí ngẫu nhiên. Delete byte. Hữu ích cho length-prefixed format.

Interesting values

Hard-coded “interesting” values: - 0 (off-by-one trigger). - 1, -1. - INT_MIN, INT_MAX, UINT_MAX. - 0x7F, 0x80, 0xFF (boundary của signed byte). - 0xDEADBEEF, 0xCAFEBAFE (magic).

Thay byte/word/dword bằng các giá trị này. Hiệu quả cao vì developer thường test happy path nhưng bug ở boundary.

Dictionary

Tập keyword đặc thù cho format. Ví dụ HTTP fuzzer: GET, POST, Content-Type, application/json, Authorization, Basic. Insert keyword vào input.

Cho format có magic header (PNG: 89 50 4E 47), dictionary đảm bảo seed luôn có magic, tránh fuzzer waste time trying random byte ở header.

Splice

Lấy 2 seed, cut tại offset ngẫu nhiên, ghép half của một với half của kia. Tạo input “trộn” tính chất 2 seed. Hữu ích khi 2 seed có 2 feature khác nhau, splice tạo input có cả hai.

Grammar-based fuzzing

Cho input structured (JSON, XML, SQL, C source code), mutation random hỏng input ngay. Cần **grammar** để sinh valid input.

Định nghĩa Grammar

Context-Free Grammar (CFG) là quy tắc sản xuất:

```
JSON      → Object | Array | String | Number | "true" | "false" | "null"
Object    → "{" Members? "}"
Members   → Pair ("," Pair)*
Pair      → String ":" JSON
Array     → "[" Elements? "]"
Elements  → JSON ("," JSON)*
String    → "\"" Char* "\""
Number    → ... (regex for number)
Char      → ... (any ASCII excluding " and \)
```

Fuzzer random walk grammar, mỗi nonterminal expand ngẫu nhiên một option.

Ví dụ output:

```
{"key1": [1, 2, {"nested": "value"}], "key2": true}
```

Valid JSON. Fuzzer test parser với input cấu trúc đúng nhưng nội dung varied.

Tool grammar fuzzer

Peach (commercial, network protocol).

Sulley: framework Python cho grammar fuzzing.

Domato (Google): JS grammar fuzzer cho browser engine. Tìm hàng trăm bug Chrome.

LangFuzz: grammar fuzzer cho JavaScript interpreter (V8, SpiderMonkey).

Csmith: grammar fuzzer cho C source code. Test compiler. Tìm 400+ bug trong GCC, Clang, LLVM.

Grammar + mutation hybrid

Modern fuzzer combine cả hai:

1. Grammar sinh seed initial valid.
2. Mutation tweak seed: flip bit ở value (giữ structure).

Tool: **Grammarinator** (combine grammar + AFL).

Black-box vs Coverage feedback

Câu hỏi: “black-box” có nghĩa “no info from target” hay “no source code”?

Definition strict: no info, không cần instrumentation. Random input + check crash. Naive Miller-style.

Definition loose (modern): không cần source code, nhưng có thể có coverage info qua binary instrumentation (PIN, DynamoRIO) hoặc shadow library.

AFL fits “loose”: cần compile với afl-gcc, nhưng không cần đọc source code. Đó là “gray-box” theo strict, nhưng cộng đồng gọi là “black-box coverage-guided”.

White-box (bài 5.7) là dùng symbolic execution, requires source-level analysis.

Khi nào dùng mutation, khi nào grammar?

Tình huống	Khuyến nghị
File format binary (PDF, PNG)	Mutation (AFL) với seed corpus đa dạng
Network protocol structured (DNS, HTTP)	Grammar hoặc mutation + dictionary
Programming language interpreter	Grammar (Domato, LangFuzz)
Library API	libFuzzer với corpus structured
Stateless function	AFL hoặc libFuzzer
Compiler	Grammar (Csmith)
Web app	Burp Suite + custom

Nếu nghi ngờ, bắt đầu với AFL/libFuzzer + good seed corpus. Quan sát coverage. Nếu stuck ở tầng parser, thêm grammar.

Tóm tắt

- **AFL** đặt nền tảng coverage-guided mutation-based fuzzing.
- Bốn ý tưởng AFL: compile instrumentation, forkserver, energy schedule, mutation strategy.
- **Grammar fuzzing** sinh input từ grammar formal, hiệu quả cho structured input.
- Modern thường **hybrid** grammar + mutation.
- Tool: AFL/AFL++, libFuzzer, Honggfuzz, Csmith, Domato.

Mini-quiz

Q1. Vì sao AFL dùng forkserver thay vì exec target mỗi test?

Exec a process tốn nhiều CPU: load ELF, resolve dynamic linker, init constructor. Trên Linux thường 5-50 ms.

Forkserver: target được init một lần, đợi command. Mỗi test case, target fork một child, child exec input, exit. Fork nhanh hơn exec (1-2 ms).

Tốc độ: AFL với forkserver chạy 1000-10000 test/giây trên 1 core. Không có forkserver: ~100 test/giây. Speedup 10-100x.

Hệ quả: fuzzing campaign hiệu quả khi run lâu (giờ/ngày). Forkserver biến hours work thành minutes.

Q2. Tại sao mutation-based fuzzing kém hiệu quả cho JSON parser nếu không có grammar?

JSON có structure rigid: bracket phải match, comma giữa elements, quote bao key/value. Bit flip random gần như chắc chắn tạo invalid JSON.

Ví dụ seed: {"key": "value"}. Flip 1 bit ở "key" thành "jey". Vẫn valid. Flip bit ở { thành 0. Invalid JSON, parser fail ngay token đầu. Bug ở deep code (sau parse OK) không được test.

Hệ quả: AFL mutation cho JSON parser thường stuck ở coverage thấp (chỉ chạm "parse failed" branch).

Giải pháp: - **Grammar**: sinh JSON valid từ grammar, mutate ở "leaf" (string content, number). - **Custom mutator**: write hàm mutate JSON-aware (insert key, change value, add nested), không phải bit flip. - **libFuzzer + structured fuzzing**: dùng LLVMFuzzerCustomMutator hook để custom mutate.

Q3. Csmith fuzz compiler thế nào? Bug tìm là loại gì?

Csmith sinh **C program** ngẫu nhiên theo grammar tinh vi (giữ C valid, không UB). Sinh hàng nghìn program, compile với GCC, Clang, ICC.

Bug tìm là **compiler miscompilation**: compiler bug khiến output binary chạy sai. Để detect:

Differential testing: compile cùng program với 3 compiler (GCC, Clang, ICC), chạy với cùng input, so output. Nếu khác $\square$ ít nhất 1 compiler có bug.

Self-test: program kết thúc với checksum của state. Nếu checksum khác giữa 3 compile, bug.

Csmith đã tìm 400+ bug trong các compiler lớn từ 2008. Hầu hết là bug optimization sai (compiler tin tưởng UB không xảy ra, optimize aggressively, kết quả sai).

Đây là ví dụ điển hình grammar fuzzing thành công: grammar formal C $\square$ valid program $\square$ differential test $\square$ compiler bug.

Tiếp theo: 5.7 White-box fuzzing (dynamic symbolic execution)

5.7 White-box fuzzing: dùng symbolic execution để sinh input

Tóm tắt một dòng: White-box fuzzing **đọc và phân tích code** để sinh input một cách thông minh. Kỹ thuật chính là **Dynamic Symbolic Execution (DSE)**, hay còn gọi **concolic execution**: chạy chương trình với input concrete, đồng thời track symbolic constraint, dùng SMT solver sinh input mới khám phá path chưa thấy.

Vì sao cần white-box?

AFL (bài 5.6) là king cho hầu hết tình huống. Nhưng có những bug **rất sâu** AFL không tìm được dù chạy hàng tuần.

Ví dụ:

```
int parse(char *input) {
    if (input[0] != 'M') return 0;
    if (input[1] != 'A') return 0;
    if (input[2] != 'G') return 0;
    if (input[3] != 'I') return 0;
    if (input[4] != 'C') return 0;
    // ... critical code ở đây, có bug ...
}
```

AFL với random mutation: xác suất hit “MAGIC” là $1/256^5 \approx 10^{-12}$. Có thể không bao giờ.

White-box: phân tích code, biết đề vào critical, input cần MAGIC. SMT solver sinh input đó trong 1 query. Tìm bug nhanh hơn 12 bậc.

Đây là động lực cho DSE.

Dynamic Symbolic Execution (DSE)

DSE còn gọi là **concolic execution** = concrete + symbolic.

Ý tưởng

Chạy chương trình với input cụ thể (concrete) như testing thông thường. **Đồng thời**, track input là symbolic variable, accumulate path constraint khi đi qua mỗi branch.

Khi chương trình kết thúc, có: 1. **Concrete trace**: chương trình đã chạy đường nào. 2. **Path constraint**: công thức SMT mô tả mọi input có cùng đường đó.

Để khám phá đường khác: 1. Phủ định một branch constraint. 2. Gọi SMT solver: tìm input thoả constraint mới. 3. Solver trả input concrete cho path mới. 4. Chạy lại với input mới, repeat.

Ví dụ chạy DSE

```
int f(int x, int y) {
    if (x > 5) {
        if (y < 10) {
            assert(x + y != 100);
        }
    }
}
```

```
return 0;
}
```

Iteration 1: chạy với input $x = 0, y = 0$.

Trace: $f(0, 0)$. Symbolic: $x = x_0, y = y_0$. - Tại if ($x > 5$): $x_0 > 5$ false. Path: $(x_0 \leq 5)$. Đi vào else (trống), return 0.

Path constraint cho path này: $x_0 \leq 5$.

Để khám phá path khác, phủ định: $x_0 > 5$. Gọi solver:

SMT: find x_0 thoả $x_0 > 5$

Solver: $x_0 = 6$

Iteration 2: chạy với $x = 6, y = 0$.

Trace: vào nhánh if ($x > 5$). Tại if ($y < 10$): $y_0 < 10$ true. Path: $(x_0 > 5) \text{ AND } (y_0 < 10)$. Đi vào assert.

Assertion $x + y \neq 100$: $x_0 + y_0 \neq 100$. Với $x_0 = 6, y_0 = 0, 6 + 0 = 6 \neq 100$. Assert pass.

Path constraint: $x_0 > 5 \wedge y_0 < 10$.

Để khám phá assertion fail, thêm constraint **phủ định assertion**: $x_0 + y_0 = 100$.

SMT: find x_0, y_0 thoả $(x_0 > 5) \text{ AND } (y_0 < 10) \text{ AND } (x_0 + y_0 = 100)$

Solver: $x_0 = 91, y_0 = 9$ (one of many solutions)

Iteration 3: chạy với $x = 91, y = 9$. Assertion fail. **Bug found.**

DSE tìm bug trong 3 iteration. So với AFL random mutation cần $\approx 10^{18}$ iteration để hit $x + y = 100$ với $x > 5, y < 10$.

DSE vs Symbolic Execution thuần

Symbolic execution (KLEE, Klee) chạy chương trình với **chỉ symbolic input**, không concrete. Tại mỗi branch, fork:

- Khám phá cả 2 nhánh: số path nổ lên (path explosion).
- Khi gặp loop, fork mỗi iteration: vô hạn path nếu loop unbounded.
- Khi gặp call ngoài (libc, syscall), không biết model như thế nào: stuck.

DSE giải quyết: - **Concrete value cho external call**: dùng giá trị concrete khi DSE đi vào libc. Không cần model. - **Path explosion controlled**: mỗi iteration chỉ chạy 1 path (concrete). Tổng số iteration = số path khám phá. - **Loop bounded**: chạy concrete tới khi exit. Không vô hạn.

DSE practical hơn symbolic execution thuần cho code thực.

SAGE: white-box fuzzer của Microsoft

SAGE (Scalable Automated Guided Execution), 2008. White-box fuzzer cho Windows binary.

Architecture

SAGE giả định binary already compiled. Dùng x86 emulator (Nirvana) để chạy concrete + track symbolic.

```

seed input → concrete run → trace với symbolic constraint
      ↓
    generational search over branches
      ↓
    for each branch:
      phủ định branch, gọi Z3
      Z3 trả input mới
      add input vào queue
      ↓
    next iteration: pick input từ queue

```

Generational search

SAGE không khám phá depth-first hay breadth-first. Dùng **generational search**:

- Generation 0: seed input.
- Generation 1: mọi input sinh từ generation 0 (phủ định mỗi branch của seed run).
- Generation 2: mọi input sinh từ generation 1 input.
- ...

Mỗi generation, pick input có “score” cao nhất (theo coverage potential, depth, etc.).

Kết quả thực tế

SAGE chạy on Windows codebase từ 2007. Đã tìm hàng trăm bug trong Office, Windows kernel, Internet Explorer. Microsoft công bố SAGE tìm “30% security bug” của Windows 7 trong giai đoạn pre-release.

Một thành công cụ thể: 2009, SAGE tìm bug parsing trong ANI cursor format, bug đã có từ Windows 95. Bug tồn tại 14 năm, AFL không tìm được, SAGE tìm trong vài giờ.

Driller: hybrid AFL + DSE

Driller (2016): combine AFL với DSE.

- AFL chạy chính, fast coverage exploration.
- Khi AFL stuck (coverage không tăng trong N giờ), invoke DSE để break through hard constraint.
- DSE sinh input vượt qua “wall”, feed về AFL.

Driller tìm bug trong DARPA Cyber Grand Challenge (2016): autonomous CTF, máy tự pwn binary.

Khó khăn của DSE

Path explosion

Mỗi branch nhân đôi số path. Chương trình có n branch có 2^n path. Với $n = 30$, đã 10^9 path, vượt khả năng explore.

Mitigation: - **Path merging**: gộp path tương tự thành 1 (giảm số state). - **State pruning**: bỏ path “không hứa hẹn”. - **Function summarization**: thay gọi function bằng summary precomputed.

External call

DSE gặp printf, malloc, system: không track symbolic được. Phải: - **Concretize**: dùng concrete value, lose symbolic info. - **Model**: viết tay model của libc function, mất công. - **Stub**: skip function, giả định

không có side effect.

KLEE có model cho hầu hết libc, vẫn limited.

Memory model

Pointer aliasing trong DSE phức tạp. $*p = 5$ với p symbolic: solver phải case split trên possible aliases. Có thể cực chậm.

Float

Symbolic float với SMT FP theory rất chậm. Hầu hết DSE concretize float, mất sound.

DSE tools

KLEE (Stanford, 2008): symbolic execution cho LLVM IR. Open source. Tìm bug trong coreutils, libc. Khoảng 50+ paper sử dụng KLEE.

SAGE (Microsoft): commercial, không open. Track record impressive trong Microsoft codebase.

Mayhem (CMU spin-off): hybrid SE + fuzz. Won DARPA Cyber Grand Challenge.

S2E (EPFL): symbolic execution cho whole-system (kernel + userland). Test driver.

angr (UCSB): Python framework cho binary analysis + symbolic execution. Popular trong CTF.

Manticore (Trail of Bits): smart contract symbolic execution (Ethereum, EVM).

Tóm tắt

- **DSE = concolic execution**: chạy concrete + track symbolic.
- Sinh input mới bằng cách phủ định branch, gọi SMT solver.
- Giải quyết external call và loop bằng concrete run.
- **SAGE** là exemplar: Microsoft dùng từ 2007, tìm hàng trăm bug.
- **Driller** hybrid AFL + DSE: AFL fast, DSE break “wall”.
- Khó khăn: path explosion, external call, memory aliasing, float.

Mini-quiz

Q1. DSE và pure symbolic execution khác nhau cốt lõi ở điểm nào?

Pure symbolic execution (KLEE original): mọi value symbolic. Tại mỗi branch, fork cả 2 path. Không có “concrete” trace.

DSE (concolic): chạy concrete trace cụ thể, đồng thời track symbolic constraint **đọc theo trace**. Mỗi iteration là 1 path concrete + 1 path constraint. Để khám phá path khác, phủ định branch, gọi solver, có input concrete mới, chạy lại.

Sự khác biệt thực dụng:

Tiêu chí	Pure SE	DSE
Path explosion	Vấn đề chính	Quản lý bằng chọn path explore
External call	Khó (cần model)	Dùng concrete value, work as-is
Loop unbounded	Vô hạn fork	Concrete run, hữu hạn
Soundness	Sound (cover all paths)	Sound trong path đã explore

DSE practical hơn cho code thực với libc, IO, syscall.

Q2. Vì sao “MAGIC” check là ví dụ kinh điển white-box thắng black-box?

```
if (input[0] != 'M') return;
if (input[1] != 'A') return;
if (input[2] != 'G') return;
if (input[3] != 'I') return;
if (input[4] != 'C') return;
// critical code here
```

Mỗi byte có 256 giá trị. Để vào critical code, cần đúng MAGIC (5 byte). Xác suất random: $1/256^5 \approx 10^{-12}$.

AFL với 10000 test/giây cần $10^{12}/10^4 = 10^8$ giây ≈ 3 năm để có 50% cơ hội hit.

White-box DSE: chạy 1 input ngẫu nhiên, fail ở byte đầu. Phủ định constraint `input[0] != 'M'`. Solver trả input có `input[0] = 'M'`. Chạy, fail ở byte 2. Phủ định, solver trả input có MA. Sau 5 iteration, có MAGIC. Vào critical code, bắt đầu fuzz deep.

DSE thắng AFL bằng cách “directed search” thay vì random. Đặc biệt mạnh khi code có nhiều check kiểu này (parser format có magic, checksum, ...).

Q3. Driller hybrid AFL + DSE thế nào? Khi nào DSE được invoke?

Driller architecture:

1. **AFL chạy chính**: fast fuzzing, scale cao.
2. **Coverage tracker**: theo dõi coverage growth.
3. **Stall detector**: nếu coverage không tăng trong N giờ, AFL stuck.
4. **DSE invoke**: feed seed corpus cho DSE engine.
5. **DSE explore**: dùng concolic để break “hard constraint” (như MAGIC).
6. **Sync corpus**: input mới DSE tạo được feed lại cho AFL.
7. **AFL tiếp tục**: với corpus richer, AFL có thể explore xa hơn.

Insight: AFL fast nhưng stupid (random mutation). DSE slow nhưng smart (solver). Combine: AFL handle 90% công việc, DSE giúp 10% còn lại (hard constraint).

Tỷ lệ time: AFL ~95%, DSE ~5%. Nhưng 5% DSE đó tìm được bug AFL không bao giờ thấy.

Đây là pattern modern fuzzing: AFL + DSE + grammar + tất cả integrate. Khả năng tự tuning trở thành quan trọng vì configuration manually rất khó.

DS perspective

Dynamic Symbolic Execution **literally** là PyTorch autograd nhưng cho boolean/integer constraint thay vì float gradient. **Symbolic variable** $\approx$ tensor với `requires_grad=True`. **Path constraint accumulation** $\approx$ backward pass accumulating gradient. **Z3 solve for input** $\approx$ gradient descent for adversarial input (FGSM, PGD). **Concolic execution** (concrete + symbolic) $\approx$ **PGD attack** (concrete starting point + gradient step). **Hybrid AFL+DSE (Driller)** $\approx$ **hybrid genetic + gradient** optimizer (CMA-ES warm-started by gradient). Bạn quen autograd = quen 80% concept symbolic execution.

Tiếp theo: 5.8 BMC for test generation

5.8 BMC for test generation: cầu nối ngược về static

Tóm tắt một dòng: BMC được thiết kế để chứng minh property holds. Nhưng nó cũng có thể được dùng **ngược** để sinh test case: encode goal coverage (ví dụ “đến được dòng X”) thành assertion ngược (“không bao giờ đến X”), BMC trả counterexample (input đến X), counterexample chính là test case. Đây là cầu nối thanh lịch giữa Lec 3-4 (static) và Lec 5 (dynamic).

Vì sao đây là bài cuối Lec 5?

Khi bắt đầu Lec 5, ta phân biệt static (Lec 3-4) và dynamic (Lec 5) như hai họ kỹ thuật khác nhau: - Static: chứng minh không bug. - Dynamic: tìm bug bằng chạy code.

Bài này show rằng phân biệt đó **không hoàn toàn rigid**. BMC, vốn là tool static, có thể được dùng để sinh input concrete, từ đó hỗ trợ dynamic testing. Đây là “elegant unification” giữa hai họ.

Bài này dạy:

1. Cách encode goal coverage thành assertion ngược.
2. Quy trình sinh test case từ counterexample.
3. Program instrumentation để track coverage trong BMC.
4. Ưu nhược so với fuzzing thuần.

Ý tưởng cơ bản

BMC trả lời câu hỏi: “có input vi phạm assertion không?”. Nếu vi phạm, trả counterexample (input cụ thể).

Đảo bài toán: muốn tìm input đi tới dòng X, viết assertion “không đi tới X”. BMC nói “có thể đi tới X” (counterexample), chính là input ta muốn.

Code minh họa:

```
int classify(int x) {
    if (x < 0) return -1;      // line A
    else if (x == 0) return 0; // line B
    else if (x < 100) return 1; // line C
    else return 2;            // line D
}
```

Muốn sinh test case đạt **statement coverage 100%**: cần test reach A, B, C, D.

Goal 1: reach A ($x < 0$)

Instrument code:

```
int classify(int x) {
    if (x < 0) {
        // GOAL 1 REACHED
        __CPROVER_assert(0, "goal_A"); // assertion luôn false
        return -1;
    }
    // ...
}
```

`__CPROVER_assert(0, "goal_A")` luôn fail nếu đến đó. BMC tìm input làm assertion fail $\square$ counterexample là input dẫn tới A.

Solver trả: $x = -1$ (hoặc số âm khác). Đây là test case 1.

Goal 2: reach B ($x == 0$)

Instrument tương tự ở line B. Solver trả: $x = 0$.

Goal 3: reach C ($x < 100$ nhưng $x \neq 0$ và $x \geq 0$)

Solver trả: $x = 50$.

Goal 4: reach D ($x \geq 100$)

Solver trả: $x = 100$.

Cuối: test suite $\{-1, 0, 50, 100\}$ đạt statement coverage 100%.

Quy trình hình thức

Cho chương trình P và tập goal $G = \{g_1, g_2, \dots, g_n\}$:

```
test_suite = []
for goal in G:
    P_instr = instrument(P, goal)  # thêm assert(0) tại goal
    result = BMC(P_instr)
    if result == SAT:
        input = extract_input(result)
        test_suite.append((input, goal))
    else:
        unreachable.append(goal)
return test_suite, unreachable
```

Output: - **test_suite**: list (input, goal) tuples. Input đi tới goal. - **unreachable**: list goal mà BMC chứng minh không reach được. Đây là **dead code** (statement không thể chạy).

Quy trình này: 1. **Tự động**: không cần human chọn input. 2. **Sound** trong bound: nếu goal reach được trong k steps, BMC tìm được. 3. **Cho ra dead code analysis miễn phí**: unreachable goal là dead code.

Program instrumentation

Chi tiết hơn về cách instrument code cho từng loại coverage criteria.

Statement coverage

Thêm `assert(0)` tại mỗi statement:

```
// Original
x = x + 1;

// Instrumented
__CPROVER_assert(0, "statement_5"); // line 5
x = x + 1;
```

Run BMC. Mỗi assertion fail tương ứng 1 test case đi tới line 5.

Branch coverage

Tại mỗi if/else, thêm assert ở cả 2 nhánh:

```
// Original
if (cond) { body_true; } else { body_false; }

// Instrumented
if (cond) {
    __CPROVER_assert(0, "branch_T_5");
    body_true;
} else {
    __CPROVER_assert(0, "branch_F_5");
    body_false;
}
```

Mỗi branch ID có 1 test case riêng.

MC/DC

Phức tạp hơn. Cần track combination của sub-condition values. Có thể implement bằng **counter variable**:

```
// Original
if (a > 0 && b > 0) { ... }

// Instrumented
bool a_pos = (a > 0);
bool b_pos = (b > 0);
__CPROVER_assert(!(a_pos == 1 && b_pos == 1), "mcdc_TT");
__CPROVER_assert(!(a_pos == 1 && b_pos == 0), "mcdc_TF");
__CPROVER_assert(!(a_pos == 0 && b_pos == 1), "mcdc_FT");
__CPROVER_assert(!(a_pos == 0 && b_pos == 0), "mcdc_FF");
if (a_pos && b_pos) { ... }
```

Solve riêng cho mỗi assert. Test cases cover MC/DC.

So sánh với fuzzing

BMC for test generation và fuzzing đều sinh test case tự động. So sánh:

Tiêu chí	BMC for test gen	Fuzzing
Coverage guarantee	Sound (đảm bảo reach nếu khả thi)	Best-effort (có thể miss)
Scale	Limited (BMC chậm cho code lớn)	Tốt (AFL chạy hàng nghìn test/giây)
Bug detection	Cần explicit assertion	Implicit (crash, sanitizer)
External code	Cần model	Chạy as-is

Tiêu chí	BMC for test gen	Fuzzing
Cost	Solver expensive	Solver-free

Best of both:

- Dùng BMC for test gen để **bootstrap corpus** cho fuzzer. BMC sinh input đi tới các deep branch khó hit bằng random.
- Sau đó fuzzer (AFL) chạy lâu với corpus đó, tìm bug runtime (memory corruption, ...).

Đây là pattern Driller (bài 5.7) generalize.

Goal coverage cho complex code

Ví dụ phức tạp hơn:

```
int validate(char *input, int len) {
    if (len < 8) return -1;
    if (input[0] != 'X') return -2;
    if (input[1] != 'Y') return -2;
    int checksum = 0;
    for (int i = 2; i < len; i++) checksum += input[i];
    if (checksum != 42) return -3;
    return 0;
}
```

Muốn test case reach return 0. Đây là goal khó cho fuzzer (cần XY ở đầu + checksum đúng).

BMC encode: $\neg len \geq 8 \vee input[0] \neq 'X' \vee input[1] \neq 'Y' \vee \text{sum}(input[2..len]) \neq 42$

Solver trả: $len = 8, input = "XY....."$ với $input[2..7]$ cộng = 42. Ví dụ $input = "XY*\backslash x00\backslash x00\backslash x00\backslash x00\backslash x00"$ (vì '*' = 42).

Test case generated. Fuzzer cần triệu iteration; BMC giải trong giây.

Path coverage

Ngoài statement/branch, BMC còn sinh test case cho **path coverage** (mọi path qua chương trình).

Số path có thể mũ (mỗi if nhân 2). Với loop, vô hạn. BMC bound số path khám phá bằng cách unwind loop.

Sinh test cho path coverage: 1. Enumerate path tree tới depth k . 2. Mỗi path p : convert thành path constraint ϕ_p (sequence of branch conditions). 3. Solve ϕ_p : input concrete cho path p . 4. Repeat.

Tool: **PathCrawler** (CEA, France) dùng BMC cho path coverage. CBMC có flag `--cover paths`.

Ứng dụng thực tế

EvoSuite (University of Sheffield): test generation cho Java. Hybrid BMC + genetic algorithm. Sinh JUnit test cho mọi method, mục tiêu coverage cao.

Pex (Microsoft Research, deprecated): white-box test gen cho .NET. SAGE technology applied to test generation.

KLEE + harness: KLEE thiết kế chính cho exploration, nhưng có thể adapt cho test gen.

CBMC với `--cover` flag: native support cho coverage-driven test gen.

KLEE + --write-test: output test case khi gặp branch mới.

Trong industry, BMC test gen thường dùng để bootstrap initial test suite, sau đó developer review và augment manually.

Hạn chế

Scale: BMC chậm cho code > vài nghìn LOC. Mỗi assertion là 1 query SMT, costs add up.

External: BMC cần model libc, syscall. Code có heavy IO khó test gen.

Path explosion: nếu cần path coverage (không chỉ statement), số path mũ.

Equality with fuzzing: cho code có nhiều UB-like bug (out-of-bounds, use after free), fuzzer + sanitizer thường tốt hơn vì detect được mà không cần explicit assertion.

Tóm tắt

- BMC for test generation: dùng BMC **ngược**, sinh input đến goal.
- Encode goal là assertion ngược (`assert(0)`) tại goal).
- BMC counterexample là test case.
- Hỗ trợ statement, branch, MC/DC, path coverage.
- Cho dead code analysis miễn phí (unreachable goal).
- Tool: CBMC, KLEE, EvoSuite, Pex.
- Best combine với fuzzing: BMC bootstrap corpus, AFL khai thác.

Mini-quiz

Q1. Vì sao gọi là “BMC ngược”? Differences với BMC for verification?

BMC for verification (chuẩn, đã thấy trong Lec 3): user viết assertion `assert(property)` để check. BMC thêm `assume(not property)` để tìm counterexample. Mục tiêu: chứng minh không có counterexample (property holds).

BMC for test generation (bài này): user định nghĩa goal (line cần reach, branch cần cover). BMC thêm `assert(0)` tại goal. Mục tiêu: tìm input dẫn tới goal (=counterexample của `assert(0)`).

Tại sao “ngược”: BMC verification mong UNSAT (proof). BMC test gen mong SAT (counterexample/input).

Cùng tool, cùng technique, hai mục tiêu trái ngược. Đây là tính linh hoạt của BMC.

Q2. Cho code:

```
int f(int x, int y) {
    if (x > 100) {
        if (y < x) {
            return x - y; // line A
        }
    }
    return 0;
}
```

Sinh test case đến line A bằng BMC test gen. Input là gì?

Encode goal: `assert(0)` tại line A.

Path constraint để đến A: $-x > 100 - y < x$

BMC + SMT solve: $-x > 100 - y < x$

Solver trả minimal: $x = 101, y = 0$ (hoặc bất kỳ tổ hợp thoả).

Test case: $f(101, 0)$ reach line A. Verify: $101 > 100$ true, $0 < 101$ true, vào A. Return $101 - 0 = 101$.

Test này cover branch A. Combined với test cho branch không đến A (ví dụ $f(0, 0)$), đạt branch coverage 100%.

Q3. Khi nào BMC test gen tốt hơn fuzzing, khi nào tệ hơn?

BMC tốt hơn khi: - Code có “hard constraint” (magic check, checksum) mà random gần như không hit. - Goal coverage là statement/branch/path specific. - Cần test case minimal (input nhỏ nhất). - Cần dead code analysis (BMC tự động phát hiện unreachable).

Fuzzing tốt hơn khi: - Code lớn (hàng vạn LOC), BMC không scale. - Bug là runtime (memory corruption, race), fuzzer + sanitizer detect tự nhiên. - Cần chạy continuously (24/7 trong CI), fuzz scale tốt. - Code có heavy IO/external call, BMC khó model.

Best practice: combine. BMC sinh seed corpus cho fuzzer (giúp fuzzer pass “wall” như MAGIC), fuzzer chạy lâu với corpus đó tìm bug runtime. Đây chính là Driller pattern (bài 5.7).

Trong industry: Google ClusterFuzz dùng fuzzer chính, plus seed corpus từ symbolic exec. Microsoft SAGE dùng cả hai. Modern fuzzing không là “AFL alone”, là một orchestration of multiple techniques.

DS perspective

BMC-for-test-gen tương tự **active learning** với **query strategy** “maximum disagreement”

- chọn input mà model uncertain nhất (high entropy). Ở đây “uncertain” = chưa cover branch.

CEGAR loop (counterexample-guided abstraction refinement) $\approx$ **iterative active learning loop**: query input mới khi gặp gap. Combine BMC + fuzzing (Driller) $\approx$ **hybrid sampling strategy**: solver giải “hard constraint” (analog của ill-conditioned region), fuzzer cover “easy region” nhanh. Cùng triết lý: smart probe cho hard case, brute force cho easy case.

Kết thúc Phần 4 (Lecture 5). Bạn đã hoàn thành toàn bộ chương trình kỹ thuật! Từ khái niệm cơ bản Software Security (Phần 1), tới BMC + SMT cho sequential (Phần 2), concurrency (Phần 3), dynamic analysis với fuzzing (Phần 4). Để thấy các kỹ thuật này áp dụng vào tư vấn dự án thực tế, đọc tiếp **Phần 5: Case Studies**.